

LaunchLens Labs — Synthetic Market Intelligence Engine

Complete Implementation Plan v2.0

Prepared for: LaunchLens Labs Dev Team

Date: April 2026

Objective: Build a production-grade Agent-Based Model (ABM) powered by diverse LLM personas to simulate district-level Indian consumer behavior with < 8% deviation from real-world outcomes.

Executive Summary

This document is the finalized technical blueprint for the LaunchLens Labs Synthetic Market Intelligence Engine. It replaces the genetic algorithm (GA) architecture described in the original pitch deck with an Agent-Based Model (ABM) driven by LLM-powered personas. The rationale is straightforward: a GA optimizes toward a single "best" solution, which destroys the population diversity that market simulation fundamentally depends on. Real consumer markets are chaotic distributions of heterogeneous behaviors, not convergent fitness landscapes. An ABM preserves that diversity while still generating predictive signal.

The plan draws directly from the *Societies.io* methodology (He et al., 2024 — British Journal of Psychology), which demonstrated that LLM-powered agents natively reproduce homophily and emergent community formation without explicit behavioral rules, and extends it with India-specific adaptations using the latest Indic LLM ecosystem (Sarvam-105B, BharatGen Param 2), Indian demographic data infrastructure (Census 2011, NCCS/ISEC classifications, NFHS), and domain-specific consumer psychology research.

The plan is organized into six phases, each with explicit goals, tasks, methodology, technical stack, deliverables, and success criteria.

Table of Contents

- 1. Architectural Philosophy: Why ABM Over GA
- 2. Phase 1: The Bharat Diversity Engine (Agent Instantiation)
- 3. Phase 2: Social Graph & Network Topology
- 4. Phase 3: Simulation Environment & Memory Architecture
- 5. Phase 4: The Interaction Loop
- 6. Phase 5: Validation & Calibration

7. Phase 6: Output Generation & Analytics
 8. LLM Routing Strategy
 9. Infrastructure & Cost Architecture
 10. Risk Register & Mitigation
 11. Development Timeline
-

1. Architectural Philosophy: Why ABM Over GA

The Problem with Genetic Algorithms for Market Simulation

The original LaunchLens deck (Slide 6) describes a pipeline where "strongest behaviors survive" and "weakest outcomes are eliminated," converging toward "reality." This is a textbook genetic algorithm. The fundamental issue is that GAs are optimization algorithms — they search for a single global optimum. Consumer markets are not optimization problems. They are complex adaptive systems with multiple coexisting equilibria, long-tail distributions, and emergent properties that arise from agent heterogeneity.

If you run a GA on a simulated population of consumers, within a few generations you get a homogeneous population of "super-buyers" — agents who all behave identically because selection pressure eliminated all the diverse, niche, or resistant behavioral profiles. You lose exactly the signal you need: which segments reject the product, which niches adopt early, which regions show unexpected resistance, and how word-of-mouth propagates through heterogeneous social networks.

The ABM Alternative

In an Agent-Based Model, agents do not evolve or die. They persist with their assigned demographic profiles and behavioral tendencies. What propagates is not "fitness" but *information* — product awareness, reviews, social proof, word-of-mouth. The simulation produces emergent macro-level patterns (adoption curves, segment clustering, regional heat maps) from micro-level agent interactions, without ever selecting for or against any particular agent type.

What Societies.io Proved

The He et al. (2024) paper, published in the British Journal of Psychology, provides the empirical foundation for this approach. Their key findings, drawn from a simulated society of 33,299 LLM-powered chatbots on the Chirper.ai platform over 28 days:

Finding 1 — Emergent Community Structure: Without any explicit social rules, LLM agents formed structurally distinct communities. Modularity scores were statistically significant (Day

28: modularity = 0.43, $p < .001$), meaning agents clustered into groups with dense internal connections and sparse external connections.

Finding 2 — Language Homophily: Communities aligned strongly with language (Day 28: $\chi^2 = 40,600.45$, Cramer's $V = 0.78$, $p < .001$). English, Chinese, and Japanese-speaking agents self-organized into separate clusters.

Finding 3 — Content Homophily: Within English-speaking agents, sub-communities formed around similar content topics. Agents posting similar content engaged more frequently (matrix permutation $r = -0.013$ to -0.024 , all $p < .001$). The effect sizes were small but consistent across all four measurement points.

Finding 4 — No Explicit Programming Required: The social behaviors were inferred by the LLMs from character prompts alone. The models were not told to prefer similar agents — they did so because homophily patterns are embedded in the training data.

How This Translates to LaunchLens

The Societies.io methodology validates that LLMs can generate realistic collective behavior from persona prompts alone. For LaunchLens, we adapt this by: replacing the social media environment with a marketplace/product evaluation environment, replacing generic character bios with demographically-grounded Indian consumer profiles, replacing the engagement metrics (likes, follows) with purchase-decision metrics (ignore, research, buy, share, complain), and adding a network propagation layer for word-of-mouth simulation.

Critical Limitation from Societies.io to Address

The paper explicitly notes that generic LLMs "may reproduce biases present in the training data" and are "likely to over-represent Western, high-income cultures." For an India-focused simulation, this is the single most important technical challenge. The mitigation strategy is a multi-model routing architecture that uses Indic-specialized LLMs for regional reasoning, detailed in Section 8.

2. Phase 1: The Bharat Diversity Engine

Goal

Create a synthetic population of consumer agents that statistically mirrors the real-world demographic distribution of a target Indian launch zone (state, district, or city level), with sufficient granularity to capture income, language, occupation, urbanization, and media consumption patterns.

Task 1.1: Demographic Data Ingestion Pipeline

Objective: Build a data pipeline that ingests, cleans, and cross-references Indian demographic

datasets to produce district-level population profiles.

Data Sources:

Source	What It Provides	Access Method
Census of India 2011 (Primary Census Abstract)	Population by district, age, sex, literacy, occupation, urbanization, SC/ST status	API at censusindia.gov.in + Kaggle mirror with geospatial indexing
NFHS-5 (National Family Health Survey, 2019-21)	Household wealth index, media exposure, smartphone ownership, education by state/district	DHS Program bulk download
NSSO Consumer Expenditure Survey	Monthly per-capita expenditure by decile, urban/rural, state	data.gov.in OGD Platform API
ISEC / NCCS Classification Data	Socio-economic classification (12-tier A1-E3) based on education + durable ownership	MRSI published distributions; Kantar/IMRB reports
TRAI Telecom Data	Mobile subscribers, internet penetration by state, rural vs urban	traai.gov.in quarterly reports
IndiaStack / UPI Data	Digital payment adoption as proxy for tech-savviness	NPCI monthly reports

Methodology:

The pipeline will normalize all data sources to a common district-level granularity (India has 766 districts as of the latest delineation). Where district-level data is unavailable (e.g., NFHS provides some metrics only at state level), the pipeline will use statistical disaggregation — applying state-level distributions to district populations weighted by urbanization rate and literacy rate from Census data.

Output: A `DistrictProfile` JSON schema for each target district containing population count, age distribution (5-year buckets), sex ratio, urban/rural split, literacy rate, language distribution (mother tongue from Census C-16 tables), ISEC/NCCS distribution, smartphone penetration estimate, and median monthly household expenditure.

Technical Stack: Python, pandas, geopandas for spatial joins, `datagovindia` PyPI package for OGD API access, PostgreSQL with PostGIS extension for spatial storage.

Dev Effort: 2 engineers, 3 weeks.

Task 1.2: Persona Generation Engine

Objective: Generate thousands of rich, natural-language persona biographies from the district-level demographic distributions, such that the simulated population is a statistically representative micro-sample of the real population.

Methodology:

This is the most critical component. The Societies.io paper showed that the quality of emergent behavior depends directly on the quality of the character prompts. Our personas must go beyond simple demographic tags to include psychographic texture that an LLM can "inhabit."

Step 1 — Stratified Sampling: For a given simulation run (e.g., simulating a product launch in Indore, MP), draw N agents (starting target: 1,000–5,000 per run) from the DistrictProfile using stratified random sampling across: age bucket, sex, urban/rural, ISEC tier, primary language, and occupation category.

Step 2 — Persona Template Expansion: Each sampled demographic vector is expanded into a natural-language biography using a persona generation prompt. The prompt combines hard demographic facts with stochastic psychographic attributes drawn from distributions calibrated to Indian consumer research.

Example Persona Output:

Name: Priya Sharma
Age: 26
Location: Indore, Madhya Pradesh (Urban, Tier-2 city)
Occupation: Junior software developer at a mid-size IT services company
Monthly Household Income: ₹45,000
Disposable Income: ~₹12,000/month
Education: B.Tech (Computer Science) from a private engineering college
Language: Hindi-dominant, consumes tech content in English, uses Hinglish on WhatsApp
Family: Lives with parents and younger brother (college student)
Media Consumption: Instagram (daily), YouTube (tech reviews, cooking), reads Dainik Bhaskar online occasionally
Shopping Behavior: Price-conscious but will pay premium for brands she trusts. Relies heavily on peer reviews and YouTube comparisons before purchases > ₹2,000. Prefers online shopping (Flipkart, Amazon) for electronics. Visits local bazaar for clothing and groceries.
Tech Adoption: Early majority – waits for friends to try new apps/products before adopting, but once convinced, becomes an active recommender.
Financial Behavior: Has a SIP (₹2,000/month), uses UPI for daily transactions, no credit card, budgets on a monthly basis.
Values: Career growth, family approval for major purchases, distrusts flashy advertising, values peer recommendations.

Step 3 — Diversity Validation: After generating the full agent population, run a statistical check comparing the simulated population's marginal distributions (age, income, urban/rural, language) against the source `DistrictProfile`. Flag and regenerate if any dimension deviates by more than 5% from the target distribution.

Key Design Decision — Why Not Fine-Tune?

Research from Toubia et al. (2025) and Arora et al. (2025) shows that augmenting LLM prompts with rich interview/profile data produces better behavioral predictions than fine-tuning. Prompt-based personas are also cheaper, faster to iterate, and don't require retraining when demographic parameters change. We use prompt engineering, not model fine-tuning, for persona instantiation.

Technical Stack: Python, Jinja2 for template rendering, a persona generation LLM (Sarvam-M or Claude Sonnet for English/Hindi bios, Sarvam-105B for regional language bios).

Dev Effort: 2 engineers, 2 weeks.

Task 1.3: Persona Quality Assurance Module

Objective: Validate that generated personas produce internally consistent behavior when queried.

Methodology: For each generated persona, run a battery of 5 "sanity check" prompts:

1. "Given your income, would you consider buying [product at ₹X]?" — Verify price sensitivity is consistent with income tier.
2. "How would you research this product?" — Verify media channel preference aligns with stated media consumption.
3. "Would you recommend this to a friend?" — Verify recommendation propensity aligns with stated adoption archetype (innovator / early majority / laggard).
4. "What language would you discuss this product in?" — Verify language output matches stated language profile.
5. "What's your primary concern about this product?" — Verify concern type (price, quality, brand trust, social proof) aligns with psychographic profile.

Personas that produce inconsistent answers on ≥ 2 of 5 checks are flagged for regeneration.

Dev Effort: 1 engineer, 1 week.

3. Phase 2: Social Graph & Network Topology

Goal

Connect agents into a social network that mirrors real-world Indian social connection patterns, enabling realistic word-of-mouth propagation, social proof effects, and information diffusion.

Task 2.1: Network Generation

Objective: Generate a social graph for each simulation's agent population using a topology that produces realistic clustering and path-length properties.

Methodology:

Use the **Watts-Strogatz small-world network** as the base topology. Small-world networks have two properties that match real social networks: high clustering coefficient (people's friends tend to know each other) and short average path length (six degrees of separation).

Parameters:

- **k (mean degree):** Set to 6–10, matching average social circle size in Indian urban/semi-urban contexts for product-relevant discussions (not total social connections, but the

subset who influence purchase decisions).

- **β (rewiring probability):** Set to 0.1–0.3 to introduce long-range connections (e.g., a person in Indore connected to a college friend in Bangalore) while preserving local clustering.

Homophily Bias: The Societies.io paper showed that homophily emerges naturally from LLM interactions. However, we also want the *initial* graph topology to reflect baseline homophily — people are more likely to know others of similar socioeconomic status and location.

Implementation: when generating the initial Watts-Strogatz ring, order agents by a composite similarity score (same district > same ISEC tier > same age bracket > same language). This ensures that the k nearest neighbors on the ring are demographically similar, while the β -rewired edges create cross-demographic bridges.

Cross-District Connections: For simulations spanning multiple districts, add a small number (2–5%) of inter-district edges weighted by geographic proximity and migration corridor data (e.g., Indore $\leftrightarrow$ Mumbai is a high-migration corridor, so more cross-connections).

Output: An adjacency list stored in a graph database (NetworkX in-memory for simulations < 10K agents, Neo4j for larger populations).

Dev Effort: 1 engineer, 1.5 weeks.

Task 2.2: Influencer Node Injection

Objective: Seed the network with a realistic distribution of high-influence nodes (local opinion leaders, micro-influencers, shopkeepers, family decision-makers).

Methodology:

Indian consumer research consistently shows that purchase decisions are heavily influenced by: family elders (especially for household goods), local shopkeepers (especially in Tier-3+ cities and rural areas), WhatsApp group forwards, and YouTube/Instagram micro-influencers.

Create special agent archetypes with higher network degree and designated influence multipliers:

Archetype	Network Degree	Influence Multiplier	Proportion
Family Elder	3-5 (family cluster)	2.0x on family connections	8-12%
Local Shopkeeper	15-25 (broad but local)	1.5x for product awareness	2-4%
Micro-Influencer	50-200 (wide, weak ties)	1.3x for awareness, 0.8x for trust	0.5-1%
WhatsApp Group Hub	10-15 (group-based)	1.2x for information spread	5-8%
Standard Consumer	6-10	1.0x	Remainder

These multipliers affect how strongly a "share" or "recommend" action from this agent type propagates to their connections.

Dev Effort: 1 engineer, 1 week.

4. Phase 3: Simulation Environment & Memory Architecture

Goal

Build the "world" that agents inhabit — including the product stimulus, the marketplace feed, and the per-agent memory system that enables multi-step reasoning and temporal consistency.

Task 3.1: Product Stimulus Package

Objective: Create a structured representation of the client's product/service that agents will evaluate.

Input from Client: Product name, category, price point(s), key features, target segment, marketing copy/creative assets, distribution channels, competitor landscape.

Stimulus Format: A structured JSON document rendered into natural language for each agent interaction:

```
json
```

```
{
  "product_name": "FreshBite Protein Bar",
  "category": "Health & Nutrition / Snacking",
  "price": {"mrp": 99, "launch_offer": 79, "currency": "INR"},
  "key_features": [
    "20g protein per bar",
    "No added sugar",
    "Available in Chocolate, Peanut, Mango flavors"
  ],
  "distribution": ["Amazon India", "BigBasket", "select modern trade outlets"],
  "marketing_copy": "Fuel your grind. India's first truly tasty protein bar.",
  "competitor_context": "Competitors: Yoga Bar (₹50-80), RiteBite Max (₹80-120), imp",
  "target_segment": "Health-conscious urban millennials, 22-35, SEC A/B"
}
```

Dev Effort: 0.5 weeks (mostly schema design + client intake form).

Task 3.2: Agent Memory System

Objective: Give each agent a persistent memory that accumulates across simulation timesteps, enabling multi-turn reasoning, opinion formation, and behavioral consistency.

Architecture:

The Societies.io paper used a simple database-backed memory where each chatbot stored its background and past actions. We extend this with a two-tier memory system:

Tier 1 — Episodic Memory (Short-term): A rolling buffer of the last 10 interactions/observations. Stored as natural language summaries. Injected into the LLM context window at each decision point.

Tier 2 — Semantic Memory (Long-term): Key facts, formed opinions, and purchase decisions stored as structured key-value pairs. Queried by relevance when constructing the agent's context.

Implementation: Use **Redis** for Tier 1 (fast read/write, TTL-based expiry) and **PostgreSQL with pgvector** for Tier 2 (structured storage + vector similarity search for retrieving relevant memories). pgvector is preferred over dedicated vector databases (Pinecone, Milvus) at this stage because it reduces infrastructure complexity and cost. Migrate to a dedicated vector DB only if query latency becomes a bottleneck above 10K concurrent agents.

Memory Schema (per agent):

```
python
class AgentMemory:
    agent_id: str
    biography: str # Immutable persona text
    episodic_buffer: List[str] # Last 10 events (natural language)
    product_awareness: Dict[str, float] # product_id -> awareness score (0-1)
    product_opinion: Dict[str, str] # product_id -> natural language opinion
    purchase_history: List[dict] # timestep, product_id, decision, reasoning
    social_interactions: List[dict] # timestep, peer_id, interaction_type, content
    opinion_embedding: np.array # Vector embedding of current opinions (for re
```

Dev Effort: 2 engineers, 2 weeks.

Task 3.3: The Marketplace Feed

Objective: Construct the environment state that each agent is exposed to at each timestep.

Design:

Adapted from the Chirper.ai feed design (the Societies.io platform), but recontextualized for commerce. At each timestep, each agent receives a "marketplace feed" composed of:

Feed Component	Source	Count
Product advertisement/stimulus	Client's product package	1 (always present when product is "live")
Peer reviews/opinions	Connected agents who have evaluated the product	Up to 5
Peer purchase decisions	Connected agents who bought/rejected	Up to 3
Competitor mentions	Randomly sampled from competitor product data	1-2
General market noise	Category-relevant context (e.g., "protein bars are trending on Instagram")	1

The feed is personalized: an agent only sees peer reviews from their direct network connections (simulating real-world information access through social circles). This is the mechanism that enables word-of-mouth propagation.

Dev Effort: 1 engineer, 1.5 weeks.

5. Phase 4: The Interaction Loop

Goal

Run the core simulation — iterating through timesteps where agents observe their feed, reason about the product, make decisions, and propagate information through the network.

Task 4.1: Decision Prompting Architecture

Objective: Design the prompt template that converts an agent's context (biography + memory + feed) into a structured decision output.

Prompt Structure:

SYSTEM: You are role-playing as the following person. Stay fully in character. Do not break character or add meta-commentary. Your responses should reflect this person's actual knowledge, biases, language patterns, and decision-making style.

CHARACTER BIOGRAPHY:

{agent.biography}

YOUR RECENT EXPERIENCES:

{agent.episodic_buffer[-5:]}

YOUR CURRENT OPINION ON THIS PRODUCT CATEGORY:

{agent.product_opinion.get(product_id, "You have not formed an opinion yet.")}

TODAY'S MARKETPLACE FEED:

{rendered_feed}

Based on your character, background, and what you've seen today, respond to the product "{product_name}" priced at ₹{price}.

1. INTERNAL REASONING: (Think through this as your character would – consider your budget, needs, what your peers said, your trust level, your priorities. Write 2-4 sentences of internal monologue.)
2. DECISION: Choose exactly one:
 - IGNORE (not relevant to you)
 - AWARE (noticed it, no action)
 - RESEARCH (want to learn more)
 - CONSIDER (seriously evaluating)
 - BUY (would purchase)
 - REJECT (evaluated and decided against)
 - SHARE_POSITIVE (would recommend to peers)
 - SHARE_NEGATIVE (would warn peers against)
 - COMPLAIN (have a specific grievance)
3. PRIMARY REASON: (One sentence explaining your decision)
4. WOULD_DISCUSS_WITH: (family / friends / colleagues / no one)

Why This Prompt Design Works:

The "internal reasoning" field is critical. The Societies.io study noted that each chatbot decision was accompanied by a "thought" generated by the LLM. In our case, these reasoning logs become the raw data for "Message Resonance" and "Objection Map" analytics — two of the key LaunchLens deliverables. By having agents articulate *why* they made each decision, we get qualitative insight at quantitative scale.

Output Parsing: The LLM output is parsed into a structured `AgentDecision` object. Use constrained generation (JSON mode or regex-based extraction) to ensure reliable parsing.

Dev Effort: 1 engineer, 2 weeks (including prompt iteration and testing).

Task 4.2: Network Propagation Engine

Objective: When an agent takes a social action (SHARE_POSITIVE, SHARE_NEGATIVE, COMPLAIN), propagate that information to their network connections for the next timestep.

Methodology:

Propagation Rules:

Agent Action	Network Effect
SHARE_POSITIVE	Agent's positive review is added to the feed of all direct connections in the next timestep
SHARE_NEGATIVE	Agent's negative review is added to the feed of all direct connections
BUY	A "peer purchased" signal (weaker than explicit review) is added to direct connections' feeds
COMPLAIN	Complaint is added to direct connections' feeds with 1.5x salience weight
IGNORE / AWARE	No network propagation

Cascade Mechanics: Information can propagate through multiple hops. If Agent A shares positive review → Agent B sees it, buys, and also shares → Agent C sees B's review. This creates realistic viral cascades. However, each propagation hop reduces salience by 30% to prevent unrealistic exponential spread.

Temporal Dynamics: Each simulation "timestep" represents approximately 1 week of real-world time. A typical simulation run covers 12–24 timesteps (3–6 months of simulated market activity). The first 2–4 timesteps model the "launch phase" where the product stimulus is first introduced.

Dev Effort: 1 engineer, 2 weeks.

Task 4.3: LLM Orchestration Layer

Objective: Manage the concurrent execution of thousands of LLM calls per timestep efficiently.

Architecture:

Use **LangGraph** as the orchestration framework. LangGraph is the current production-grade choice for stateful, graph-based multi-agent workflows (GA in May 2025, used in production by ~400 companies). It provides built-in support for: state management across agent turns, conditional routing (e.g., routing Hindi-persona agents to Sarvam-105B vs. English personas to Claude Sonnet), parallel execution (scatter-gather pattern for batch agent processing), and checkpointing for simulation resumability.

Execution Strategy:

For a 5,000-agent simulation with 12 timesteps = 60,000 LLM calls per run.

- **Batch Processing:** Process agents in batches of 100–500, using async HTTP calls to LLM APIs.
- **Rate Limiting:** Respect API rate limits with exponential backoff. Target throughput: 50–100 agents/second.
- **Estimated Wall-Clock Time:** 10–20 minutes per timestep, 2–4 hours per full simulation run.
- **Cost per Run:** At ~500 tokens input + 200 tokens output per agent per timestep, using Sarvam-105B API (estimated \$0.001–0.003/call) = \$60–\$180 per 5K-agent, 12-timestep run.

Dev Effort: 2 engineers, 3 weeks.

6. Phase 5: Validation & Calibration

Goal

Prove that the simulation produces outputs that match real-world market data with < 8% deviation, and build the systematic calibration process that makes this claim defensible to investors and clients.

Objective: Build a library of 10–15 well-documented Indian product launches with known outcomes, to use as ground truth for calibration.

Methodology:

Select launches across your three target verticals (FMCG, D2C, SaaS) where the following data is available or can be sourced: launch city/region, launch price point, target demographic, actual adoption curve (units sold or active users over first 6 months), demographic breakdown of early adopters (from post-launch surveys or analytics), and marketing spend/channels.

Recommended Calibration Products (starting points):

Vertical	Product Example	Why It's Good for Calibration
FMCG	Paper Boat (beverage) national expansion	Well-documented Tier-2/3 expansion, nostalgia-driven marketing, clear demographic data
D2C	mamaearth early growth phase	D2C with strong social media + trust-based buying, public revenue data available
SaaS	Zoho regional adoption	Indian SaaS with documented SMB adoption patterns
FMCG	Raw Pressery cold-pressed juices	Premium pricing in a price-sensitive market — good test of rejection modeling
D2C	boAt earbuds launch phase	Youth-oriented, tech product, heavy influencer marketing

Data Sourcing: Combination of published case studies, industry reports (RedSeer, Redseer Strategy, Inc42 archives), public filings (for listed companies), and if budget permits, purchase syndicated data from Nielsen/Kantar for 2–3 key cases.

Dev Effort: 1 researcher + 1 engineer, 3 weeks.

Task 5.2: Calibration Protocol

Objective: Systematically compare simulation outputs against historical ground truth and tune system parameters.

Methodology:

For each calibration product:

1. **Configure the simulation** with the product's actual launch parameters (price, city, target segment, marketing channel).
2. **Run the simulation** for the product's known launch duration.
3. **Compare outputs** against ground truth on five metrics:

Metric	Measurement	Target Deviation
Overall Adoption Rate	% of target population that reached BUY state	< 8%
Adoption Curve Shape	DTW (Dynamic Time Warping) distance between simulated and real curves	< 0.15 normalized DTW
Top Segment Accuracy	Does the simulation identify the same top 3 demographic segments as real data?	≥ 2 of 3 match
Regional Heat Map	Rank correlation between simulated city-level adoption and real data	Spearman $\rho > 0.7$
Rejection Reason Alignment	Do the top 3 simulated rejection reasons match post-launch survey data?	≥ 2 of 3 match

4. **Tune parameters** based on deviations:
 - If adoption rate is too high → increase price sensitivity weight in persona prompts
 - If adoption rate is too low → increase social proof multiplier
 - If wrong segments are adopting → audit persona generation for demographic accuracy
 - If adoption curve is too fast → reduce propagation speed / increase timestep duration
5. **Cross-validate** by holding out 3 calibration products as a test set, calibrating on the rest, and measuring test-set performance.

Dev Effort: 2 engineers, 4 weeks (iterative).

Task 5.3: Bias Detection & Mitigation

Objective: Identify and correct systematic biases in LLM agent behavior, particularly Western/affluent bias and "sycophantic" positivity bias.

Known Biases to Test For:

1. **Affluence Bias:** LLMs trained on internet data over-represent affluent, English-speaking, urban perspectives. Test by comparing simulated SEC-D/E agent behavior against known low-income consumer research.
2. **Positivity Bias:** LLMs tend to be "helpful" and may over-generate BUY decisions. Test by comparing simulated rejection rates against known category-level rejection rates.
3. **Language Bias:** Even with Indic LLMs, code-mixed (Hinglish) reasoning may not match actual Hindi-belt consumer reasoning patterns. Test with focus group comparison.
4. **Homogeneity Bias:** All agents from the same demographic band may produce too-similar responses. Test by measuring response variance within cohorts.

Mitigation Strategies:

- For affluence bias: Use Sarvam-105B for low-ISEC agents, with prompts explicitly grounding budget constraints.
- For positivity bias: Add a "skepticism prior" to prompts for Indian consumers (research shows Indian consumers are generally more price-skeptical than Western consumers).
- For language bias: Validate a sample of regional-language agent reasoning against human raters from that region.
- For homogeneity bias: Add controlled stochastic variation to persona prompts (e.g., randomizing within $\pm 15\%$ of income band, varying personal values).

Dev Effort: 1 engineer + 1 domain expert, ongoing (2 weeks initial, then continuous).

7. Phase 6: Output Generation & Analytics

Goal

Transform raw simulation logs into the eight LaunchLens deliverables shown in the pitch deck (Slide 7).

Task 6.1: Analytics Pipeline

Objective: Build automated pipelines that process simulation logs into each deliverable.

Deliverable Mapping:

Deck Deliverable	Raw Data Source	Processing Method
Market Fit	Agent decision distribution (BUY / REJECT / IGNORE)	Aggregate by demographic segment, compute adoption rate per segment
City Intelligence	Agent decisions grouped by simulated district	Geographic heat map with adoption rate per district
Adoption Curve	Cumulative BUY decisions per timestep	Time-series plot with confidence intervals from multiple simulation runs
Segment Depth	Agent decisions clustered by behavioral pattern	K-means clustering on agent decision vectors, label clusters
Message Resonance	Agent "internal reasoning" logs	NLP analysis: extract mentioned marketing claims, sentiment per claim, frequency
Feature Priority	Agent reasoning referencing specific features	Count feature mentions in BUY vs REJECT reasoning, compute feature importance score
Objection Map	Agent reasoning for REJECT and COMPLAIN decisions	Topic modeling (BERTopic) on rejection reasons, ranked by frequency
Scenario Analysis	Multiple simulation runs with varied parameters (price, city, segment)	Side-by-side comparison tables with delta analysis

Task 6.2: Report Generation

Objective: Auto-generate a client-facing report from the analytics output.

Format: Interactive web dashboard (primary) + downloadable PDF report (secondary).

Dashboard Tech: React frontend with Recharts for visualization, Mapbox for geographic heat maps. Backend: FastAPI serving analytics results from PostgreSQL.

Dev Effort: 2 engineers, 3 weeks.

8. LLM Routing Strategy

Goal

Route each agent's LLM calls to the most appropriate model based on the agent's language profile, socioeconomic tier, and the task being performed.

Routing Table

Agent Profile	Primary LLM	Fallback	Rationale
English-dominant, SEC A/B, Urban	Claude Sonnet 4 via API	GPT-4o-mini	Best reasoning quality for premium segment; training data well-represents this demographic
Hindi-dominant or Hinglish, any SEC	Sarvam-105B API	Sarvam-M (24B)	Sovereign Indic model, trained on 12T tokens including Indian languages, supports romanized/code-mixed text
Regional language (Tamil, Telugu, Bengali, Marathi, etc.)	Sarvam-105B (supports 22 Indian languages)	BharatGen Param 2	Best available regional language reasoning; Apache 2.0 license allows self-hosting
Rural, SEC D/E, low literacy	Sarvam-105B with specialized low-income prompt template	Sarvam-M with constrained vocabulary	Needs explicit budget grounding; generic models over-represent urban affluent behavior
Persona Generation (batch)	Claude Sonnet 4	Sarvam-M	High-quality, diverse persona text generation
Reasoning Analysis (post-hoc)	Claude Sonnet 4	—	Best at extracting themes from agent reasoning logs

Cost Optimization

Sarvam-105B activates only ~9B parameters per token via MoE architecture, making inference cost significantly lower than dense 100B+ models. For the bulk of agents (Hindi/regional), this is the cost-efficient choice. Reserve Claude/GPT calls for English-dominant agents and post-hoc analysis tasks.

Estimated LLM Cost Distribution per 5K-agent run:

- 60% of agents → Sarvam-105B API: ~\$40-80
- 25% of agents → Claude Sonnet API: ~\$30-60
- 15% of agents → Sarvam-M (self-hosted on rented GPU): ~\$10-20
- Post-hoc analysis: ~\$15-30
- **Total estimated LLM cost per run: \$95-\$190**

9. Infrastructure & Cost Architecture

Compute Infrastructure

Component	Service	Monthly Cost Estimate
Orchestration Server	AWS EC2 c6i.2xlarge (8 vCPU, 16GB)	\$250
PostgreSQL + pgvector	AWS RDS db.r6g.large	\$200
Redis (episodic memory)	AWS ElastiCache r6g.large	\$150
Graph DB (Neo4j, if >10K agents)	Neo4j Aura Professional	\$200
LLM API Costs (10 runs/month)	Sarvam + Claude + misc	\$1,000–2,000
Dashboard Hosting	Vercel / AWS Amplify	\$50
Monitoring & Logging	Datadog or Grafana Cloud	\$100
Total Monthly Infrastructure		\$1,950–\$2,950

First-Year Budget Alignment with Fundraise

Raise Allocation	Amount	How This Plan Uses It
Model R&D (40%)	\$400,000	LLM API costs, calibration dataset acquisition, bias testing, prompt engineering team (2 ML engineers × 12 months)
Product Infra (20%)	\$200,000	Cloud infrastructure, dashboard development, DevOps (1 infra engineer × 12 months)
City Expansion (25%)	\$250,000	District-level data acquisition for 50+ cities, regional validation studies, regional language testing
Customer Acquisition (15%)	\$150,000	Sales, marketing, pilot program subsidies

10. Risk Register & Mitigation

Risk	Likelihood	Impact	Mitigation
LLM agents produce unrealistically homogeneous behavior	High	Critical	Stochastic persona variation, diversity metrics per cohort, temperature tuning
Sarvam-105B API has availability issues or rate limits	Medium	High	Multi-model fallback chain; pre-negotiate enterprise SLA or self-host Sarvam-M
Calibration cannot achieve < 8% deviation	Medium	Critical	Start with higher-data verticals (FMCG); accept 10-12% for v1 and tighten iteratively; be transparent with investors about which verticals have validated deviation
Indian demographic data is stale (Census 2011)	High	Medium	Cross-reference with NFHS-5 (2021), TRAI quarterly data, and projected population estimates from Census projections
Agents "break character" and produce generic LLM responses	Medium	High	Character consistency checks (Task 1.3), prompt engineering with explicit stay-in-character instructions, post-hoc filtering of out-of-character responses
LLM cost per run exceeds projections	Medium	Medium	Implement aggressive caching (identical personas seeing identical feeds can share LLM calls), use smaller models for low-stakes decisions (IGNORE/AWARE)
Client expects 100% accuracy, not statistical prediction	High	Medium	Clear deliverable framing: "statistically most likely outcome" not "prediction"; include confidence intervals in all outputs

11. Development Timeline

Phase-Gate Timeline (18 weeks to MVP)

Week	Phase	Milestone	Gate Criteria
1-3	Phase 1	Demographic pipeline + first 1,000 personas generated for Indore	Personas pass diversity validation check

Week	Phase	Milestone	Gate Criteria
4-5	Phase 2	Social graph generated, influencer nodes placed	Graph metrics (clustering coefficient, avg path length) match small-world benchmarks
6-8	Phase 3	Memory system, feed architecture, product stimulus schema complete	End-to-end single-agent test: one agent receives feed and produces valid decision output
9-11	Phase 4	Full interaction loop running, 100-agent test simulation	100 agents complete 6 timesteps without errors; network propagation produces observable cascades
12-14	Phase 5	Calibration against 3 historical products	≥ 1 product achieves $< 10\%$ deviation on adoption rate
15-17	Phase 6	Analytics pipeline + dashboard MVP	All 8 deliverables auto-generated from simulation logs
18	Integration	First full 5,000-agent simulation run + client-ready report	End-to-end pipeline produces defensible output

Post-MVP (Months 5-12)

- Expand calibration dataset to 15+ products across FMCG, D2C, SaaS
- Tighten deviation target from $< 10\%$ to $< 8\%$
- Add scenario analysis (price sensitivity sweeps, A/B copy testing)
- Expand to 50+ Indian districts
- Build API for Agency tier (white-label integration)
- Patent filing for the multi-model persona routing architecture

Appendix A: Key Academic References

1. He, J.K., Wallis, F.P.S., Gvirtz, A., & Rathje, S. (2024). "Artificial intelligence chatbots mimic human collective behaviour." *British Journal of Psychology*. DOI: 10.1111/bjop.12764 — Primary methodological foundation.
2. Chu, M.L. et al. (2025). "LLM-Based Multi-Agent System for Simulating and Analyzing Marketing and Consumer Behavior." *IEEE ICEBE 2025*. arXiv:2510.18155 — Validates LLM agents for consumer discount response simulation.

3. **Toubia, O. et al. (2025).** "Consumer Digital Twins" — Demonstrates that prompt-augmented LLMs outperform fine-tuned models for individual behavior prediction.

4. **Arora, N., Chakraborty, I., & Nishimura, Y. (2025).** "AI-Human Hybrids for Marketing Research." *Journal of Marketing*, 89(2), 43-70. — Population-level synthetic user validation.

5. **Park, J.S. et al. (2023).** "Generative Agents: Interactive Simulacra of Human Behavior." *ACM UIST 2023*. — Foundational generative agent architecture (memory, reflection, planning).

6. **Sarvam AI (2026).** Sarvam-105B Technical Documentation. — 105B parameter Indic MoE model, 22 languages, Apache 2.0.

Appendix B: Glossary

Term	Definition
ABM	Agent-Based Model — simulation paradigm where autonomous agents interact to produce emergent macro behavior
GA	Genetic Algorithm — optimization technique that evolves populations toward a fitness optimum
ISEC	Indian Socio-Economic Classification — 2024 update to NCCS, 12-tier system based on income, education, occupation
NCCS	New Consumer Classification System — predecessor to ISEC, based on CWE education + durable ownership
SEC	Socio-Economic Classification — original 1988 Indian consumer classification (deprecated)
MoE	Mixture of Experts — neural architecture where only a subset of parameters activate per token, reducing inference cost
Homophily	Tendency for individuals to associate with similar others — the key emergent behavior validated by Societies.io
DTW	Dynamic Time Warping — algorithm for measuring similarity between two temporal sequences
pgvector	PostgreSQL extension for vector similarity search
LangGraph	Open-source Python framework for graph-based multi-agent orchestration (by LangChain, MIT license)

This document is version 2.0 of the LaunchLens Labs implementation plan. It supersedes the genetic algorithm architecture described in the original pitch deck. The plan is designed to be handed directly to a development team for execution.